

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

SESSION 1

Getting Acquinted with GPU Programming

Groothuis Thomas (r0955783)
De Smet Feniks (r0891378)

Academic year 2025–2026

Contents

1 Problem & Goals **2**

2 Methodology **2**
2.1 Design 2
2.2 Implementation 2

3 Results **3**

4 Conclusion **4**

A CUDA Implementation **4**

1 Problem & Goals

This lab addresses the problem of reversing a one-dimensional array using GPU parallelism. To efficiently exploit data-level parallelism, the workload is distributed across a grid of thread blocks, each containing multiple threads. The primary objective is to design and execute a parallel array-flipping algorithm on the GPU. This process entails managing device memory allocation, handling data transfers between the host and device, and verifying the correctness of the final output.

2 Methodology

2.1 Design

We designed the algorithm to flip each element of the array individually. To make this possible, each element on the array has to be mapped to a unique thread on the GPU. To do this we let each thread calculate the global index using the following formula:

$$\texttt{idx} = \texttt{threadIdx.x} + \texttt{blockIdx.x} \times \texttt{blockDim.x} \tag{1}$$

The basic idea of flipping an array is to place the first item of the input array into the last position of the output array. This logic is applied to every position in the input array by correspondingly decreasing the index of the output array. Consequently, if `idx` represents the index of the input array for a given thread, the index of the output array is calculated as:

$$\texttt{output index} = \texttt{n} - \texttt{idx} - \texttt{1} \tag{2}$$

2.2 Implementation

We start off the code with the necessary includes. The kernel function we wrote is called `vecFlip` and it takes 3 arguments: the pointers to the input & output arrays, as well as an integer `n` that specifies the length of the array. The kernel calculates the **idx** for each thread, after which it checks if **idx** is less than `n`. This bounds check is necessary because the GPU launches threads in fixed-size blocks, meaning the total number of threads often exceeds the array size `n`. This check prevents these extra threads from accessing out-of-bounds memory. If **idx** is within bounds, the element at index **idx** in the input array will be put at index `n` - **idx** - `1` of the output array.

In the `main` function, the program first verifies that exactly three command-line arguments are provided. If not, it prints the message `"Usage: ./vecflip <array_size> <num_blocks>"` and terminates execution. Accepting the array size `N` and the number of blocks as arguments allows for easy testing of multiple grid configurations without recompiling the code. Once parsed, `N` is used to allocate and initialize the host arrays, with the input array sequentially populated from 0 to `N` - 1.

Next, device memory is allocated using `cudaMalloc`. We allocate memory for both the input array `d_a` and the output array `d_b`, ensuring the size is specified in bytes (`N * sizeof(int)`). The initialized input data is then transferred from the host memory to the device memory using `cudaMemcpy` with the `cudaMemcpyHostToDevice` flag. To configure the execution grid, the program dynamically calculates the required `threadsPerBlock` based on the defined number of blocks. This is achieved using integer ceiling division: `(N + blocks - 1) / blocks`. This formula guarantees that the total number of launched threads (`blocks` $\times$ `threadsPerBlock`) is always greater than or equal to the array size `N`, ensuring every element is processed.

To accurately measure the performance of the parallel execution, CUDA events (`cudaEvent_t`) are utilized. A start event is recorded immediately before launching the `vecFlip` kernel. The kernel is invoked using the dynamically calculated execution configuration, specifying the number of `blocks` and `threadsPerBlock`. Because CPU and GPU executions are asynchronous, `cudaDeviceSynchronize()` is called to explicitly halt the host process until the GPU completes all operations. Afterward, a stop event is recorded and synchronized. The elapsed execution time is then computed using `cudaEventElapsedTime` and printed to the console.

Once the kernel execution concludes, the resulting flipped array is transferred back from the device to the host memory using `cudaMemcpy` with the `cudaMemcpyDeviceToHost` direction flag. To validate the correctness of the GPU implementation, a sequential CPU loop iterates through the output array `b`. It verifies that each

element `b[i]` matches the expected corresponding value from the original array at index `N - i - 1`. Finally, the program performs necessary memory management cleanup to prevent memory leaks: device memory is freed using `cudaFree`, dynamically allocated host memory is released using `delete[]`, and the CUDA events are destroyed.

3 Results

Size N	Block Count	Threads per Block	Correct Result	Time (ms)
10	2	5	True	0.085
256	2	128	True	0.087
4096	2	2048	False	0.077
4097	2	2049	False	0.076
5000	2	2500	False	0.079
5001	2	2501	False	0.081
10	4	3	True	0.093
256	4	64	True	0.109
4096	4	1024	True	0.087
4097	4	1025	False	0.075
5000	4	1250	False	0.077
5001	4	1251	False	0.074
10	8	2	True	0.097
256	8	8	True	0.084
4096	8	512	True	0.086
4097	8	513	True	0.087
5000	8	625	True	0.083
5001	8	626	True	0.088

Table 1: CUDA vector reversal performance results

First, it must be noted that if the calculated number of threads per block exceeds the hardware limit of 1024, the program fails to provide a valid solution. As demonstrated in the results table, any configuration requiring more than 1024 threads per block (e.g., a configuration of 2 blocks with 2500 threads each) results in a silent kernel failure. Consequently, no computation takes place on the GPU, yielding an invalid output array populated entirely with zeros.

For valid configurations, an analysis of the execution times reveals that computation time generally decreases as the degree of parallelism increases. By distributing the workload across multiple threads, the GPU's Streaming Multiprocessors (SMs) are more effectively utilized, processing array elements concurrently rather than sequentially. However, the time required to flip the array is bound to the time required to flip one item of the array. Because we generally reserve at least `N` threads over the `N-sized` array, the time required is equal to the time of flipping 1 item (distributed over `N` threads). However, slight fluctuations or exceptions in this trend can be observed. These anomalies are primarily caused by kernel launch overhead, memory transfer latencies, and hardware saturation, where the base time required to schedule the threads outweighs the actual computation time, especially for smaller array sizes.

Interestingly, the data also demonstrates that excessively increasing the number of blocks leads to a higher average execution time. This performance degradation is a direct consequence of the underlying GPU hardware architecture. When a large number of blocks is defined for a fixed array size `N`, the dynamically calculated number of threads per block drops significantly. Because the GPU hardware schedules and executes threads in *warps* of 32, a block containing fewer than 32 threads still consumes a full warp, leaving the remaining threads entirely idle. This leads to severe under-utilization of the computational resources and introduces significant block scheduling overhead, thereby increasing the total execution time.

4 Conclusion

We can conclude that CUDA is a great tool for parallelization, given that the user is aware of the hardware and software limits. This means that if we want to operate more threads simultaneously, and a block has reached its maximum capacity of 1024 threads, we must scale the operation by increasing the amount of blocks.

A CUDA Implementation

```cpp
#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib>

__global__ void vecFlip(int *a, int *b, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        b[n - idx - 1] = a[idx];
    }
}

int main(int argc, char* argv[]) {

    if (argc != 3) {
        std::cout << "Usage: ./vecflip <array_size> <num_blocks>" << std::endl;
        return 1;
    }

    int N = atoi(argv[1]);
    int blocks = atoi(argv[2]);

    // Dynamische host allocatie
    int *a = new int[N];
    int *b = new int[N];

    int *d_a, *d_b;

    // Initialize host arrays
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));

    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Minimale threads per block berekenen
    int threadsPerBlock = (N + blocks - 1) / blocks;

    std::cout << "Array size: " << N << std::endl;
    std::cout << "Blocks: " << blocks << std::endl;
    std::cout << "Threads per block: " << threadsPerBlock << std::endl;

    // ---- CUDA TIMING ----
    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);

    cudaEventRecord(start_cuda);

    vecFlip<<<blocks, threadsPerBlock>>>(d_a, d_b, N);

    cudaDeviceSynchronize();

    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);

    float ms = 0.0f;
    cudaEventElapsedTime(&ms, start_cuda, stop_cuda);

```

```
    std::cout << "Kernel execution time: " << ms << " ms" << std::endl;

    // Copy back
    cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);

    // Correctness check
    bool correct = true;
    for (int i = 0; i < N; i++) {
        if (b[i] != a[N - i - 1]) {
            correct = false;
            break;
        }
    }

    std::cout << "Correct result: " << (correct ? "True" : "False") << std::endl;

    // Cleanup
    cudaEventDestroy(start_cuda);
    cudaEventDestroy(stop_cuda);
    cudaFree(d_a);
    cudaFree(d_b);
    delete[] a;
    delete[] b;

    return 0;
}
```